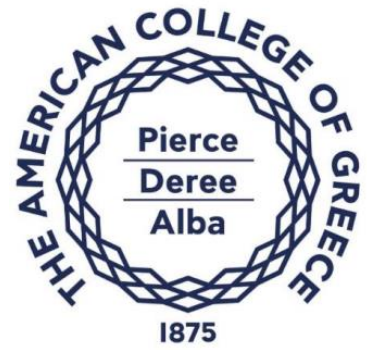




Real-Time Chess Tracking System

**A Machine Vision
Project for Chess
Game Detection and
Validation**

**Panagiotis Kounelis
Ioannis Zouras
Konstantinos Megaritis
Ifigenia Mavraki
Shalva Bakradze**

Table of Contents

Cover Page	1
Abstract.....	4
Keywords	4
Introduction	5
Materials & Methods.....	6
1. Apparatus & Assumptions	6
Assumptions:	6
Apparatus.....	6
Libraries	7
2. Function Definitions.....	8
2.1 Helper Functions.....	8
2.2 Draw Chessboard Functions	10
2.3 Move Validation Functions	11
2.4 Checkmate Check Functions	13
3. Variable Definitions.....	14
Results & Discussion	15
Results.....	15
1. Chessboard Calibration	16
2. Piece Initialization	18
3. Game in Progress.....	20
4. End Game	24
Discussion	24
Conclusion.....	26
Inclusion of other End Game conditions	26
Clock Functionality	26
Additional expansions.....	27
References	28

Table of Figures

Image 1: Apparatus used for the live video feed.....	7
Image 2: 8x8 2D array used to initialize the virtual chessboard	9
Image 3: Visualization of virtual chessboard presented to end user at the piece initialization phase	11
Image 4: Stacked visualization window, in order from left to right: Canny edge detection, two round of dilation, 2nd largest connected component, bounding box used for frame warping. Top of the image shows the trackbars for threshold calibration. First row shows Median blurring results, while bottom one shows those of Gaussian blurring	17
Image 5: Warped and overlayed live video feed of the chessboard with pieces placed in position	18
Image 6: Histogram of the difference image after application of band thresholding – all pixels concentrated in black and white values, 0 and 255 respectively	20
Image 7: Steps occurring behind the scenes for a simple move to be detected. In order: piece move by user on physical chessboard, grayscale difference image, band threshold difference image and finally eroded and dilated difference image.....	21
Image 8: Example where user (white pieces), performs a common chess habit of recentering several of their pieces on the physical board. This results in a scenario of more than 4 non-black components detected by the algorithm and the user will message to reset	22
Image 9: Camera not aligned with chessboard example	24

Abstract

The field of machine or computer vision has revolutionized technology, enabling machines to detect, interpret, and analyze their environment with remarkable accuracy. This project explores its application in developing a real-time system that detects and tracks chess games through a mobile phone webcam feed. The system our team has developed offers an automated solution for monitoring and validating chess moves by leveraging the computational power of OpenCV and combining it with the strategic complexity of chess. It overcomes the limitations of manual processes and expensive equipment by providing an affordable alternative that utilizes readily available tools like webcams and smartphone cameras. The system processes chessboard data dynamically, recognizing and validating moves by chess rules while delivering real-time feedback. Its operation is structured into a series of phases: chessboard calibration to accurately define the playing area, piece initialization to determine initial positions using image segmentation, move tracking through advanced image comparison techniques to identify piece movements and move validation to ensure compliance with chess rules. Additionally, the system identifies game-ending scenarios, summarizes moves, and records their accuracy. The project uses OpenCV and Python to incorporate sophisticated image processing techniques such as edge detection, thresholding, dilation-erosion, connected components and image transformation. Despite some challenges like inconsistent lighting, the application of strict setup assumptions helped us achieve the desired outcome. The integration of our collaborative efforts with machine vision principles and applications exemplifies the significant potential of computer vision. This approach enables the development of versatile and innovative solutions for precise tracking and validation, particularly valuable in chess and other domains requiring high levels of accuracy.

Keywords

Chessboard detection, Move validation, Image & Video Processing, Machine vision techniques, Gameplay automation

Introduction

In recent years, computer vision has been considered a revolutionary technology that allows machines to interpret and understand their environment. This project seeks to evaluate the application of computer vision by designing a system that uses a web camera to recognize and observe a chess game in progress. The integration of the functionalities of OpenCV and the nature of the game of chess enables the system to offer a semi-automated and effective means of game monitoring and verification. The main aim of this project is to create a setup that handles chessboard actions in live time, correctly spots and checks moves based on set chess rules and gives picture updates for better understanding.

There are four phases that the system is iterating through while the game is in process. The first phase, Chessboard Calibration, is aimed at identifying the game area. The second phase, Piece Initialization aims to help user set up the pieces in the squares' centers. Third phase, Game in Progress, essentially the main execution of the algorithm where move tracking, chess rules validation and end-game scenario assessment are being performed. Finally, the fourth phase, End Game, where the player can either quit or save game states.

Implementing a chess tracking system in real time comes with its own set of challenges, such as: alterations in the lighting, the possibility of other external objects blocking the camera, or differences in the physical construction of chess pieces. Challenges such as these require in solution the use of complex techniques. Bringing chess to life in such a way demonstrates how visual data processing works together with logical processes. Though the focus is on chess, this method could be used in other board games, in sports and other activities that require accurate position tracking and confirmation.

Materials & Methods

This section provides a look at the apparatus used as well as a detailed breakdown of the algorithm's functional components, functions, which will be deployed in the main execution phase, which we will explore further in the Results & Discussion section.

1. Apparatus & Assumptions

Assumptions:

1. The camera is positioned directly above the chessboard.
2. The camera is placed to the right side of the player controlling the white pieces.
3. The camera's angles are perfectly aligned with the corners of the chessboard.
4. The gray value of the background is not equal to the gray value of the chessboard.

Apparatus

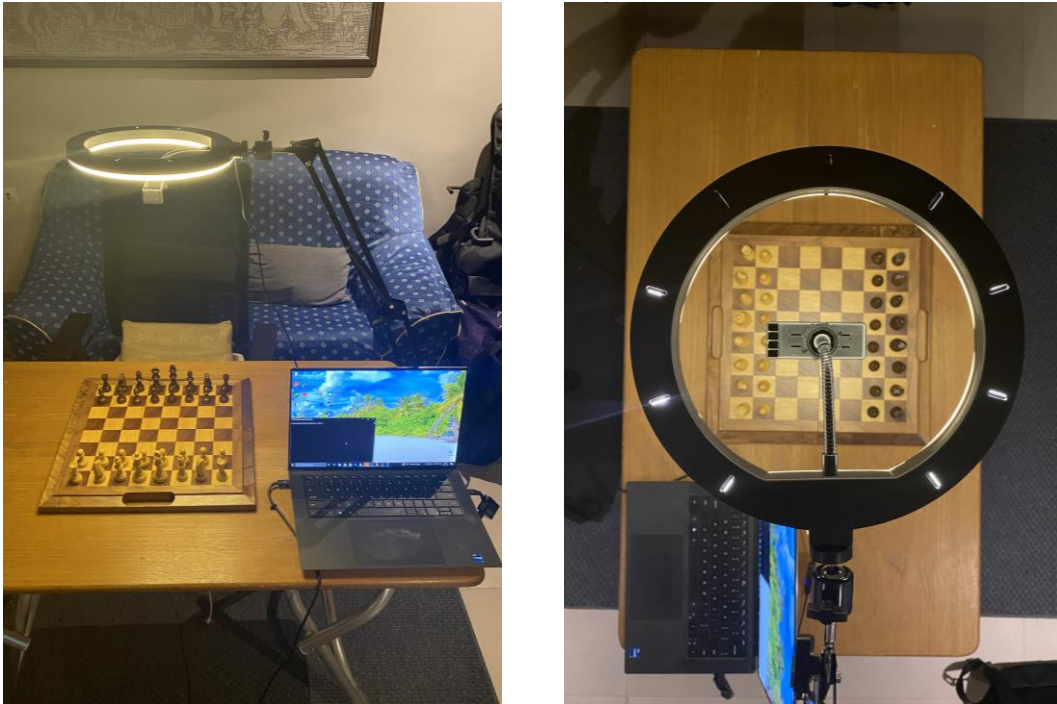
The physical apparatus used for the assumptions compliance was a ring light, a phone, a laptop and a chessboard with pieces.

The system is connected to a live webcam that allows it to observe the moves made by the players.

For the live video feed display, EpocCam was the mobile application of choice, which required the download and installment of Elgato Camera Hub PC application and a mutual internet network for connectivity.

The implementation phase starts with placing a mobile phone on an edge of the chessboard at an angle of ninety degrees in order to take clear high-quality pictures and videos unobstructed by other external factors.

Image 1: Apparatus used for the live video feed



Libraries

The libraries imported and used in this algorithm are the below:

1. **OpenCV** – Image processing
2. **NumPy** – Matrix operations and logical manipulation
3. **copy** – Array deep copy creation
4. **Counter** – Array comparison
5. **os** – File handling for saving game states
6. **Python standard libraries** – Logical implementation for validation and game logic

2. Function Definitions

Multiple functions are defined in the algorithm, to reduce repetitive coding, improve readability and break it down to smaller, more easily managed and debugged components.

2.1 Helper Functions

These functions help mainly with underlying positional changes during the chess game.

nothing:

Used as a place holder.

show_image_with_waitkey(image, window_name = 'Image'):

Showing images with waitKey(0) and then destroying all windows, while accepting two parameters: a 2D NumPy array image and the subsequent name of the popped window in a string format, i.e. "".

is_point_in_mask(mask, point):

Checking whether a point is within a specified binary mask (typically NumPy array), accepting two parameters mask and point.

position_to_indices(square):

Converting chess coordinates to corresponding board indices, by accepting one parameter; square. In the example of 'B3', B indicates the column and will be converted to 1, as the column conversion is a zero-based index (0-7). As for the row index 'B3' would translate to 5, as it is calculated by subtracting the square's number from 8. So 'B3' would equal to (5,1).

indices_to_position(row, col):

Converting board indices to chess coordinates, essentially achieving the opposite effect to **position_to_indices()**, by accepting two parameters row and columns, eg. (5,1) will be converted to 'B3'.

paint_centers_and_squares(image, squares_centers_dict):

Painting a red circle (cv2.circle) at the center of squares and annotating (cv2.putText) the chess square label near it, by accepting a NumPy array image and a dictionary that has chess square labels as keys and corresponding center coordinates as respective values. Finally, the function returns painted and annotated images.

get_piece_at_square(board_matrix, square):

Retrieving location of a piece in the chessboard matrix, by accepting two parameters: the chessboard matrix as a 2D array and a specific chess coordinate, eg. 'B3'. By utilizing the previously defined **position_to_indices()** function, 'B3' is converted to board indices, (5,1) and finally the value of the position is returned.

initialize_chessboard():

Initializes chessboard, where each element represents a piece on the board, whereas None indicates an empty square.

Image 2: 8x8 2D array used to initialize the virtual chessboard

```
chessboard = [  
    ["bR1", "bN", "bB", "bQ", "bK", "bB", "bN", "bR2"], # Black major pieces  
    ["bP", "bP", "bP", "bP", "bP", "bP", "bP", "bP"], # Black pawns  
    [None, None, None, None, None, None, None, None], # Empty row  
    [None, None, None, None, None, None, None, None], # Empty row  
    [None, None, None, None, None, None, None, None], # Empty row  
    [None, None, None, None, None, None, None, None], # Empty row  
    ["wP", "wP", "wP", "wP", "wP", "wP", "wP", "wP"], # White pawns  
    ["wR1", "wN", "wB", "wQ", "wK", "wB", "wN", "wR2"], # White major pieces  
]
```

move_piece(board_matrix, start, end):

Essentially achieves movement of a piece from one chess square to another. This is achieved by utilizing some of the predefined functions. In more detail, initially, **get_piece_at_square()** retrieves the piece at the starting position. Following that **position_to_indices()** is called twice to get the starting and end positions. Finally, an if statement is used to validate if there is a piece at the starting position, if there is no piece in the starting position, an f-string is used, `print(f"No piece at {start} to move.\n")`, to output to the user that there is no piece at said position. On the other hand, if there is indeed a piece in the starting position the piece is placed to the end position and removed from the starting one.

2.2 Draw Chessboard Functions

Four main functions were defined to help create and visualize a virtual chessboard for the end user.

draw_chessboard_with_labels(custom_light_color = (181, 217, 240), custom_dark_color = (99, 136, 181)):

Creating a visualization of a virtual chessboard and labelling each square in traditional chess labelling manner, i.e. A-H for columns and numbers 1-8 in descending order for rows. The formation of the chessboard is achieved through three for loops: first loop creates the grid, second adds the row numbers and third the column letters.

add_pieces_from_matrix(board, board_matrix):

Overlaying chessboard with pieces based on the chessboard matrix. Pieces are imported from PNG files from a specified file path, resized (`cv2.resize`) and placed to appropriate location on the board.

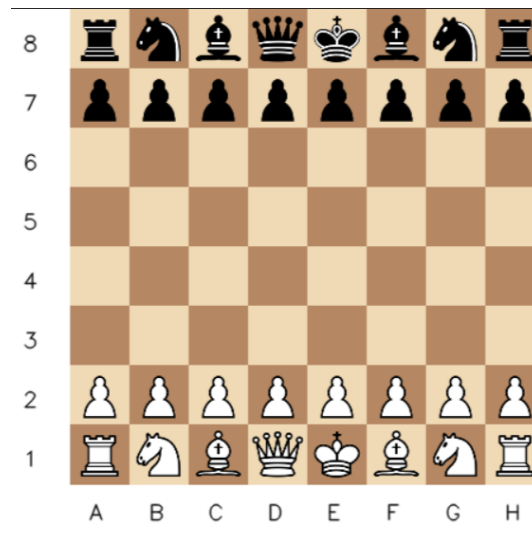
overlay_image(board, piece_image, x, y):

Achieves transparency of the background found on PNG images containing the pieces added from **add_pieces_from_matrix()**, essentially blending the added pieces seamlessly with the chessboard.

visualize_chessboard_with_labels(board_matrix):

Visualizes virtual chessboard through creating a global variable, in which the **draw_chessboard_with_labels()** is stored and finally by calling the **add_pieces_from_matrix()**.

***Image 3:** Visualization of virtual chessboard presented to end user at the piece initialization phase*



2.3 Move Validation Functions

The following functions help establish the rules of chess to assess whether moves performed by the player/s are valid or not, including special moves like pawn promotion, en passant, castling, etc. All above is done while simultaneously verifying that the moves do not create check for the king.

is_valid_move(board_matrix, start, end, player_color):

Assessing whether a move is valid or not based on specific type of piece and how it moves, i.e. assessing start and end position, by using if, elif logical operators and calling piece specific validation functions listed below:

validate_pawn_move(board_matrix, start, end, player_color)

validate_rook_move(board_matrix, start, end)

validate_knight_move(start, end)

validate_bishop_move(board_matrix, start, end)

validate_queen_move(board_matrix, start, end)

validate_king_move(board_matrix, start, end, player_color)

All above functions validate moves by calling **position_to_indices()** function followed by chess rules applied with simple calculations, logical operators and loops.

pawn_promotion(board_matrix, end):

Executes pawn promotion by asking the end user to choose what should the pawn be promoted to:

- Press Q key for Queen
- Press R key for Rook
- Press B key for Bishop
- Press N key for Knight
- If any other key is pressed the end user receives an “Invalid choice. Please choose again” prompt.

Behind the scenes, the piece is identified by calling the **get_piece_at_square()** and then verified that it is indeed a pawn. Inside a nested if statement it is assessed whether the pawn is on the end_row by calling **position_to_indices()** and then the local variable, initially set to None, is converted to a piece of the user's choice.

validate_en_passant(board_matrix, start, end):

Assesses whether it is valid for a pawn to capture an opponent's pawn that has moved two squares in the previous move. Behind the scenes, start and end positions are captured by calling **position_to_indices()**, inside a nested if statement several conditions are defined that if met, the global variable `en_passant_flag` is converted to True, allowing the en passant capture.

validate_castling(board_matrix, player_color, rook_id):

Assesses whether castling is allowed by validating that the king or rook have not moved thus far and if that is the case the algorithm continues to validate that there are no pieces between the specific rook and the king. In the code this is achieved with several logical conditions that if met the global variable `castling_flag` is converted to True, allowing castling.

is_square_under_attack(board_matrix, square, player_color):

Checks whether a piece is under attack by another piece, differentiating between own and opponents' pieces.

2.4 Checkmate Check Functions

Below functions work together to detect a check situation, whether there are potential exit strategies and if not a checkmate.

find_king_square(board_matrix, king_color):

Locates the king's square for a specific `king_color`, by scanning the chessboard through a nested for loop and returning the coordinates.

is_king_in_check(board_matrix, king_color):

By combining function call of both **find_king_square()** and **is_square_under_attack()**, this functions achieves to check whether the king is in check.

has_legal_moves(board_matrix, king_color):

Through a combination of logical operators and function calling of **indices_to_position()**, this function achieves to check for legal moves by the player, i.e. it simulates moves where the king is not in check.

is_checkmate(board_matrix, king_color):

By combining with functions in below way:

is_king_in_check() and not has_legal_moves()

inside an if statement, this function achieves to assess whether there is a checkmate situation.

3. Variable Definitions

In this section of the algorithm we defined all necessary variables, lists and dictionaries that were needed later in the main execution section. The purpose was to have all of them clearly defined in one section, in order to improve readability and organization of our algorithm. We hope that this step will facilitate better understanding from individuals who may not be familiar with the structure of our algorithm.

Results & Discussion

Results

Main execution section is where the algorithm starts getting executed and it is divided into four key phases:

1. Chessboard Calibration – Setting up the virtual chessboard through real world input through the live feed video
2. Piece Initialization – Placing physical pieces on the board for the algorithm to identify positions and apply accordingly on virtual chessboard
3. Game in Progress – The game begins and the algorithm is actively managing gameplay and move validation
4. End Game – Conclusion of the game either via checkmate, resignation or tie

A list, named **phases**, containing all the phases of the game is created. Following that comes the creation of an iterator object assigned to **phase_iter**, which allows sequential access to each phase.

At the start of the main execution stage a variable named **phase** is defined as can be seen below, which retrieves the first phase, Chessboard Calibration, and assigns it to **phase**.

```
phase = next(phase_iter)
```

The final step before entering the Chessboard calibration phase is to create an infinite loop using a while statement, for live video feed until stopped manually or interrupted. In more detail, video is captured with the object `cap.read()` from OpenCV library which is then displayed by `cv2.imshow()` in a pop up window named “webcam”.

1. Chessboard Calibration

By utilizing a while loop, the algorithm ensures the phase remains in `phases[0]`, i.e. chessboard calibration, until the calibration process is completed.

Once again, while in this phase the live video feed is captured and displayed, i.e. showing the chessboard in real-time, with `cv2.imshow()`.

Video is converted to grayscale through OpenCV's `cv2.cvtColor(,COLOR_BGR2GRAY)`.

Median blurring is applied on the gray-scaled frame, through `cv2.medianBlur(, 5)` using a 5x5 kernel in order to reduce noise. Median blurring uses a kernel to iterate through an image pixel by pixel, where it replaces each value with the median value of the neighborhood of pixels within the size of the kernel (Bradski, G., & Kaehler, A., 2008). Additionally, we compared the effects of blurring with the application of Gaussian blurring, and concluded that median blurring was more fitted to our needs. Comparison can be seen in image 5.

Canny edge detection is applied to track the edges of the frame, by detecting areas with a rapid change of intensity. Canny uses multiple steps to achieve the latter, some notable being gradient computation, non-maximum suppression & edge tracking by hysteresis (Canny, J., 1986). Trackbars for low and high thresholds are also utilized to optimize the edge detection of the frame.

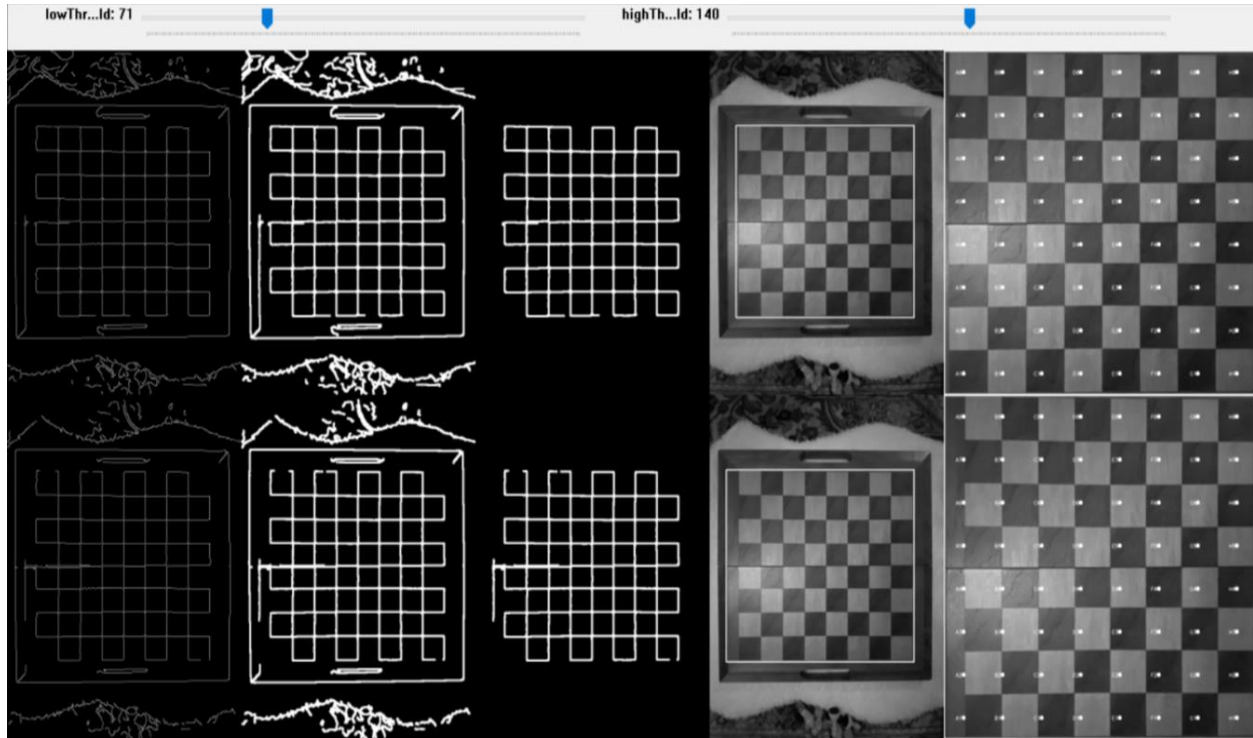
Following edge detection two rounds of dilation are applied to thicken the slim edges produced from Canny edge detection.

Connected component analysis with Stats is applied to the dilated frame and the largest non-black (not background) component is selected. By extracting the statistics of that component from the stats array, its area is calculated, and a bounding box is drawn around it by using `cv2.rectangle()`.

The bounding box's coordinates are used to align the image in the perspective transformation that follows. Specifically, the chessboard is warped to an 800x800 frame, assuming the camera is positioned directly on top of the chessboard, as stated in the assumptions section.

All above, except median blurring, are visualized in a multiple video stacked pop out window, which allows calibration through trackbars that control low and high thresholds.

Image 4: Stacked visualization window, in order from left to right: Canny edge detection, two round of dilation, 2nd largest connected component, bounding box used for frame warping. Top of the image shows the trackbars for threshold calibration. First row shows Median blurring results, while bottom one shows those of Gaussian blurring



Once the chessboard is calibrated, the user is prompted to “Set up chessboard with initial piece positions. Press SPACE when ready to start the game.\n”. Pressing the SPACE key will cause the iterator to object to move the next phase, destroy all calibration windows and break the while loop that was initiated in the start of the phase.

It is important to note that once the SPACE key is pressed and the initialization phase is over, neither the chessboard nor the camera should move until the end of the game.

2. Piece Initialization

Once again, by utilizing a while loop, the algorithm ensures the phase remains in the current phase, phases[1] in this case, until the piece initialization process is completed. In addition, the live video feed is captured and displayed continuously, i.e. showing the chessboard in real-time, with cv2.imshow().

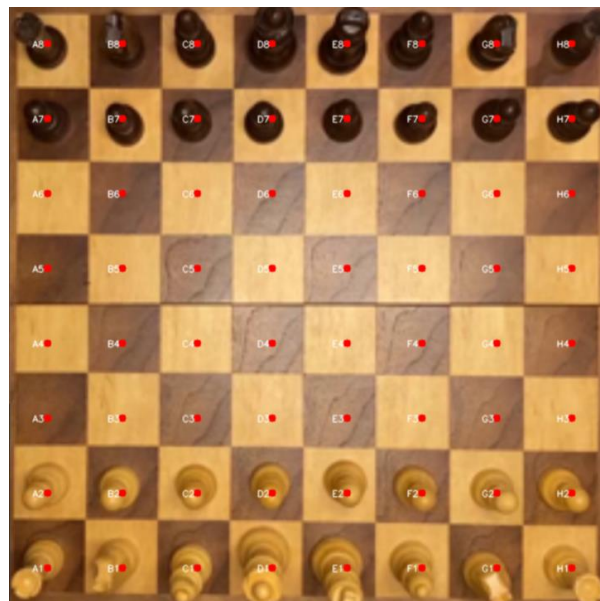
Through cv2.warpPerspective(), the bounding box from the previous phase is used and the live video feed gets a perspective transformation, i.e. is warped to an 800x800 frame.

By using dst.copy(), a snapshot copy of the chessboard frame is created without any annotations for future steps.

The warped frame is then overlayed with red centers and annotations for each square, by calling the previously defined **paint_centers_and_squares()**.

Then the warped and overlayed chessboard is displayed to the user, essentially showing the centers of each square to assist in the correct and accurate positioning of pieces by the user/s.

Image 5: Warped and overlayed live video feed of the chessboard with pieces placed in position



Once the all the pieces are positioned, the user has to press the SPACE key to start the game.

Once the user presses the SPACE key, a series of events get triggered behind the scenes:

1. The iterator object moves to the next phase.
2. The system outputs “Game started” & “White to move”, so that users can commence the game.
3. The functions **initialize_chessboard()** and **visualize_chessboard_with_labels()** get called to visualize the virtual chessboard matrix to the user/s.
4. A list, named game_baords, is initiated with the current chessboard configuration to log board states during the game.
5. Takes a grayscale snapshot of the starting chessboard state.
6. Breaks the while loop that was initiated in the start of the current phase.

3. Game in Progress

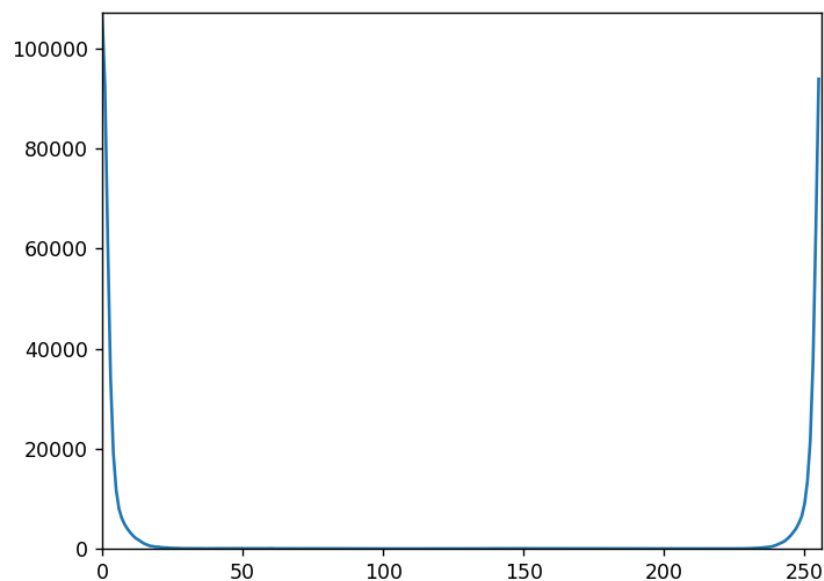
Once again, by utilizing a while loop, the algorithm ensures the phase remains in the current phase, phases[2] in this case, until the game ends by one of the following ways Checkmate situation, user presses “D” key for draw or “Q” key for resigning. As in previous phases, the warped and annotated live video feed is displayed continuously as well as the virtual chessboard matrix, in which moves occurring in the physical chessboard are translated into it.

The game starts and for each move to be finalized the user/s has to press the SPACE key, for it to be validated, checked for checkmate situations and finally for the virtual chessboard matrix to be updated.

In detail, once the SPACE key is pressed the following events occur behind the scenes inside an if statement:

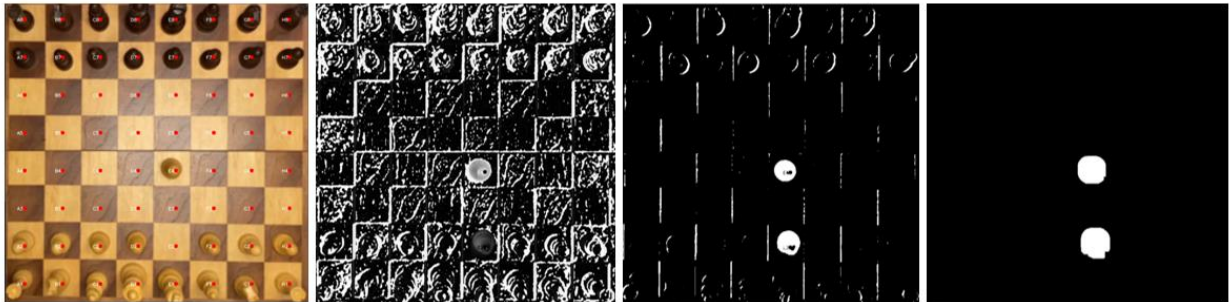
1. A grayscale snapshot is taken from which the previous snapshot is subtracted, stored in the diff variable, essentially being the difference of the two. Band thresholding is applied, cv2.inRange(), in the difference image.

Image 6: Histogram of the difference image after application of band thresholding – all pixels concentrated in black and white values, 0 and 255 respectively



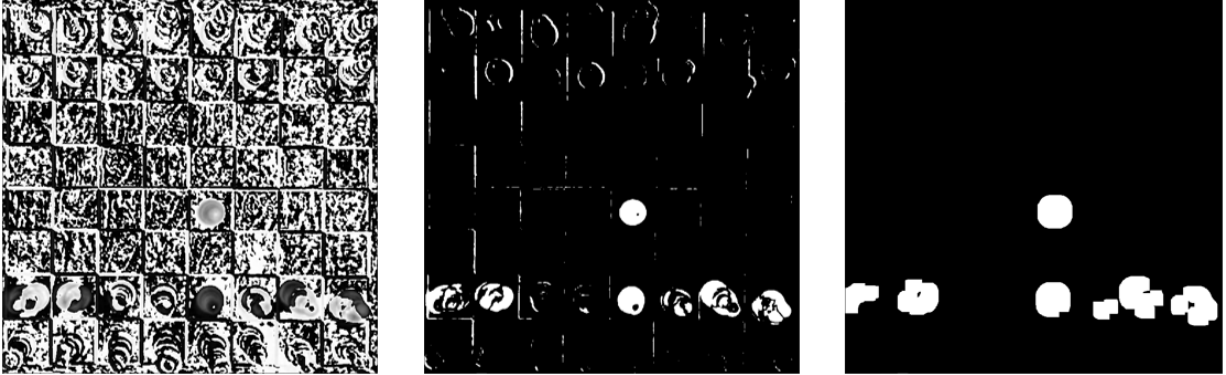
2. Four rounds of erosion are applied to reduce/remove noise.
3. More rounds of dilation than erosion follow, twelve to be specific, in order to emphasize the start and end positions in the difference image.
4. Connected component analysis with Stats is applied on the dilated difference image. Depending on the components identified on the image, using logical operators one of the below outcomes occurs:
 - Only 1 component, i.e. background – User gets message “Make a move before pressing SPACE.”.
 - Only 1 non-black component – No move corresponds to only one item being white and user gets the message "Only 1 change detected. Reset board and play again."
 - 2 non-black components – Simple move detected, algorithm gets a pass statement and moves to next line of code

Image 7: Steps occurring behind the scenes for a simple move to be detected. In order: piece move by user on physical chessboard, grayscale difference image, band threshold difference image and finally eroded and dilated difference image



- 3 non-black components – Possible en passant move. Algorithm validates later whether the move is valid or not according to chess rules
- 4 non-black components – Possible castling move. Algorithm validates later whether the move is valid or not according to chess rules
- More than 4 non-black components – User gets the following message: "Too many changes detected. Reset board and play again."

Image 8: Example where user (white pieces), performs a common chess habit of recentering several of their pieces on the physical board. This results in a scenario of more than 4 non-black components detected by the algorithm and the user will message to reset



5. Following the connected component analysis of the pieces moves, the dynamically updated variables are being reset to avoid move detection and validation overlap from previous moves or errors. More specifically, the `dynamic_vars` dictionary is cleared, `square_not_found_flag` is reset to `False` and `involved_pieces` is cleared. Since the algorithm heavily depends on accurate frame analysis, this step ensures that the data provided is isolated and solely from current move.
6. Through a defined range for-loop resulting from the number of connected components found in step 4 and by utilizing the `dynamic_vars` dictionary the non-black components are masked and subsequently the start and end squares are detected.
7. Given the squares detected the piece/s involved are identified and depending on the number of non-black connected components number the move categorized as simple move, en passant or castling. If no mask overlaps with a square center then the user gets the following message "Some mask does not overlap with any square center. Reset board and play again."
8. The move then gets validated by calling the **`is_valid_move()`** function and whether the move results in a illegal move that puts the king in a check by calling the **`is_king_in_check()`** function. If move is indeed valid the user/s get an f-string message `f"Moved {moving_piece} from {start_square} to {end_square}\n"`, essentially annotating the moves in a script. If the move is invalid the following message gets displayed "Invalid move. Reset board and play again."

9. The move then gets visualized in the virtual chessboard matrix and a snapshot of the move gets appended to the `game_boards` array initialized in the peace initialization phase.
10. Move is assessed whether it creates a checkmate situation for the opponent by calling the **`is_checkmate()`** function and if that is the case the f-string message `f"Checkmate! {current_turn_color.upper()} wins!\n"` gets displayed. If not, the `previous_snapshot` variable is updated with the current state of the chessboard to be used in the next move detection, specifically the subtraction step and the game continues by updating the turn variables. The user/s get the message "White to move" or "Black to move".

In addition to the SPACE key, the user can press the "R" key to reset the previous snapshot. This is needed for example in a case like that presented in image 9, where the user/s will be presented with message "Too many changes detected. Reset board and play again.". Upon resetting the chessboard, they will have to press the "R" key to return to the previous state of the game and essentially clear the dynamic variables for a fresh start. Effectively, it is a manual override to ensure accurate and consistent move detection in case of potential errors.

This could be useful in a situation where a player would press SPACE key while his/her hand is still on the board, resulting in an additional component being the hand.

As mentioned previously the game can end by one of the following ways

- 1. Checkmate situation**
- 2. User presses "D" key for draw**
- 3. User presses "Q" key for resigning**

Any of these will result in the iterator object moving the next phase, i.e. `phases[3]` or End Game phase.

4. End Game

Entering the End game phase all windows are destroyed for both the live video feed and the virtual chessboard. Then the user has the below two options:

1. Press the “S” key to save all captured 2D snapshots of the moves inside the game_boards array to a predefined file path.
2. Press the ESC key to simply exit the game.

Discussion

While effective under controlled conditions and strict assumptions, the algorithm can be affected by multiple conditions such as camera placement, lighting, errors in the key pressing from users, which highlight that there is room for further optimization.

The camera’s strict placement was intentionally selected as a different angle of the camera causes the following issues:

1. **Camera not aligned with chessboard:** Largest component border edges in chessboard calibration phase following connected components analysis, will not align with the actual chessboard.

Image 9: Camera not aligned with chessboard example



2. **Camera being not above the chessboard:** Pieces will be hiding other pieces and part of the chessboard, which would most probably result in warping issues in cases where the edges of the chessboard are not clearly visible. In the case of an AI expansion for piece recognition, there could be malfunctions of the algorithm caused by unclear visual data for each piece.

In addition to the camera's position, the lighting of the board was controlled, as it was observed that natural light coming from different directions, produced further noise and shadows of the pieces, which ended up producing more components than anticipated in the subtraction step of the game in progress phase.

Conclusion

The algorithm captures live video from a camera, while simultaneously displaying it to the user together with a virtual chessboard. It uses machine vision techniques to process the chessboard in real-time. It validates moves, simple and advanced, according to chess rules, visualizes updates and handles endgame scenarios including checkmate, resignation or a tie.

Furthermore, there is potential for extensions to improve the algorithm's experience and functionality. Some such ideas are described shortly below for potential future exploration.

Inclusion of other End Game conditions

Draw by Stalemate – Scan the entire chessboard matrix by using the **is_valid_move()** function and given that no piece has a valid move, including the king, i.e. if he moves it leads in a check situation, which could be assessed by the **is_king_in_check()**, and if both conditions are met, result in a stalemate. In that case a message can be presented to the user/s “No legal moves available. Draw by stalemate. \n\n Press S key to save game moves or ESC key to quit game. \n\n Thank you for playing!”.

Other end game conditions include fifty-move rule and threefold repetition.

Clock Functionality

Incorporating a clock functionality would enhance the game's realism. Additionally, since several game modes can be implemented it would expand the users' game options, keeping users more engaged.

This could be achieved by importing the time library module, initiating a timer for each player, according to selected game mode, and recording the time the game began and subtracting from each player's initially defined timer. If the timer reaches 0 for a player, a message will be displayed f“Time's up! {player_color} has run out of time. \n\n{opponent_color} wins the game! \n\nPress S key to save game moves or ESC key to quit game. \n\n Thank you for playing!”. At the same time by deploying OpenCV's cv2.putText() the timer of each player could be overlayed in the live feedback window.

Additional expansions

Some additional expansions could include multi-angle camera support, integration with AI for piece recognition, integration of separate AI for game analysis-move suggestion, expansion to other board games and finally different gameplay modes such vs bot or vs online player.

To conclude, the algorithm provides a functional framework on which improvements and expansions could be added in the future, which could pave the way for a more versatile, immersive and enjoyable chess-playing experience for users.

References

1. **Bradski, G., & Kaehler, A. (2008).**
Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
2. **OpenCV Documentation**
Available at: <https://docs.opencv.org>
3. **Canny, J. (1986).**
A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, *PAMI*-8(6), 679–698.
4. **TutorialKart (n.d.).**
Gaussian Image Smoothing Using OpenCV. Available at:
<https://www.tutorialkart.com/opencv/python/opencv-python-gaussian-image-smoothing/>
5. **TutorialKart (n.d.).**
Image Edge Detection Using OpenCV. Available at:
<https://www.tutorialkart.com/opencv/python/image-edge-detection/>
6. **PyImageSearch (2021).**
OpenCV Thresholding (cv2.threshold). Available at:
<https://pyimagesearch.com/2021/04/28/opencv-thresholding-cv2-threshold/>